INFORMATICS INSTITUTE OF TECHNOLOGY

INFORMATICS INSTITUTE OF

TECHNOLOGY DEPARTMENT OF

BUSINESS MANAGEMENT

DOC325 – Digital Circuits and Logic Design

COURSEWORK – 2022/23
(Foundation Jan 2023)

Lecturer:	Ms. Aathika Salam / Ms. Chathuri Udagedara
Team No:	C-1
Submission Date:	23rd of July 2023

Student Id	Student Name
20221804	Muhammed Saad Mazhar
20221803	Dilina Chathuraka Perera
20221802	Rafeek Mohammed Alfar
20221798	Chadithya Dinuwan Udayantha
20222336	Mohamed Zabrin Mohamed Juzail
20222330	Akash Sathiyanathan

Acknowledgment

We extend our heartfelt appreciation to all those who made this report possible. We would like to offer special recognition to our module leader, Ms. Aathika Salam, and Ms. Chathuri Udagedara for their unwavering guidance and support throughout this project. Our profound gratitude goes to Ms. Aathika Salam and the other dedicated Digital Circuit and Logic Design lecturers, as they have been instrumental in imparting the knowledge essential for completing this document. Therefore, we would like to dedicate a section of this text to express our gratitude towards them.

Furthermore, we are grateful to all the inspiring IIT lecturers who have contributed to our growth and assisted us on this journey. We would also like to extend our thanks to our parents and friends for their invaluable support in all our endeavors.

Introduction

Train Departure Safety System

The safety and efficiency of train departure processes are of paramount importance in modern railway systems. To ensure a secure and seamless boarding experience for passengers while maintaining strict adherence to safety protocols, a sophisticated Train Departure Safety System has been implemented. This report aims to provide an overview of the system's design, outlining its inputs, outputs, and the rationale behind its implementation.

The Train Departure Safety System is deployed at railway stations, where it plays a pivotal role in monitoring critical conditions before authorizing train departures. The system operates at the platform level and acts as a guardian to ensure the safety of passengers, personnel, and the train itself during the boarding process. It leverages real-time data and precise sensors to make informed decisions, minimizing the risk of accidents and ensuring a smooth departure for trains.

The Train Departure Safety System plays a pivotal role in ensuring passenger safety and operational efficiency within the railway infrastructure. By integrating intelligent decision-making based on the inputs received, the system exemplifies the commitment to providing a secure and reliable transportation experience for all rail passengers.

The implementation of the Train Departure Safety System is crucial for various reasons. Firstly, it significantly reduces the possibility of accidents during the boarding process, safeguarding the well-being of passengers and staff alike. Secondly, the system optimizes train departure efficiency by providing a clear departure signal when all safety conditions are met, streamlining the boarding process. Moreover, the real-time monitoring of safety concerns through the safety alert (Y) output ensures timely responses to any potential hazards.

Description of the inputs and the outputs

Inputs:

1. Platform Availability (A) :

    - This input takes the value '1' if the platform is unavailable for any reason, such as ongoing maintenance or an unforeseen obstruction, otherwise '0' when the platform is accessible.

2. Train Arrival Status (B) :

    - The second input is '1' when the train has arrived at the station, and '0' at all other times when the train is not present.

3. Passenger Request to Board (C) :

    - This input is '1' when passengers are actively boarding the train, and '0' when there are no boarding requests.

4. Door Safety Condition (D) :

    - The final input takes the value '1' when the train doors are open for boarding, and '0' when they are securely closed.

Outputs:

5. Departure Signal (X) :

    - This output is '1' when the platform is available, the train has arrived, boarding requests have ceased, and the train doors are safely open, indicating a departure is authorized. Otherwise, the output is '0'.

6. Safety Alert (Y) :

    - This output is '1' when the train doors are open, signifying a potential safety concern. When the doors are closed and all safety conditions are met, the output is '0'.

Truth table for above scenario

A	B	C	D	A’	C’	C.D	B.C’	A’D (Y)	B.C’.D.A’ (X)	Minterm	Maxterm
0	0	0	0	1	1	0	0	0	0	A’.B’.C’.D’	A+B+C+D
0	0	0	1	1	1	0	0	1	0	A’.B’.C’.D	A+B+C+D’
0	0	1	0	1	0	0	0	0	0	A’.B’.C.D’	A+B+C’+D
0	0	1	1	1	0	1	0	1	0	A’.B’.C.D	A+B+C’+D’
0	1	0	0	1	1	0	1	0	0	A’.B.C’.D’	A+B’+C+D
0	1	0	1	1	1	0	1	1	1	A’.B.C’.D	A+B’+C+D’
0	1	1	0	1	0	0	0	0	0	A’.B.C.D’	A+B’+C’+D
0	1	1	1	1	0	1	0	1	0	A’.B.C.D	A+B’+C’+D’
1	0	0	0	0	1	0	0	0	0	A.B’.C’.D’	A’+B+C+D
1	0	0	1	0	1	0	0	0	0	A.B’.C’.D	A’+B+C+D’
1	0	1	0	0	0	0	0	0	0	A.B’.C.D’	A’+B+C’+D
1	0	1	1	0	0	1	0	0	0	A.B’.C.D	A’+B+C’+D’
1	1	0	0	0	1	0	1	0	0	A.B.C’.D’	A’+B’+C+D
1	1	0	1	0	1	0	1	0	0	A.B.C’.D	A’+B’+C+D’
1	1	1	0	0	0	0	0	0	0	A.B.C.D’	A’+B’+C’+D
1	1	1	1	0	0	1	0	0	0	A.B.C.D	A’+B’+C’+D’

SOP and POS for the outputs

Y output

SOP –

(A’.B’.C’.D)+ (A’.B’.C.D)+(A’.B.C’.D)

POS –

(A+B+C+D).(A+B+C’+D).(A+B’+C+D).(A’+B+C+D).(A’+B+C’+D).(A’+B’+C+D’.
(A’+B’+C’+D).(A’+B’+C’+D’)

X output

SOP –

(A’.B.C’.D)

POS –

(A+B+C+D).(A+B+C+D’).(A+B+C’+D).(A+B+C’+D’).(A+B’+C+D).(A+B’+C’+D).(A+B’+C’+D’).
(A’+B+C+D).(A’+B+C+D’).(A’+B+C’+D).(A’+B+C’+D’)
.(A’+B’+C+D).(A’+B’+C+D’).(A’+B’+C’+D).(A’+B’+C’+D’)

Simplified answer using k-map and Boolean laws

Using k-map

Using Boolean laws

Y output:

A’B’C’D+A’B’CD+A’BC’D
A’B’D’(C’+C)+A’BC’D – By distributive law
A’B’D1+A’BC’D – By complement law
A’B’D+A’BC’D – By identity law
A’D(BC’+B) – By distributive law
A’D(C’+B’) – By absorption law
A’DC’+A’DB’ – By distribution law

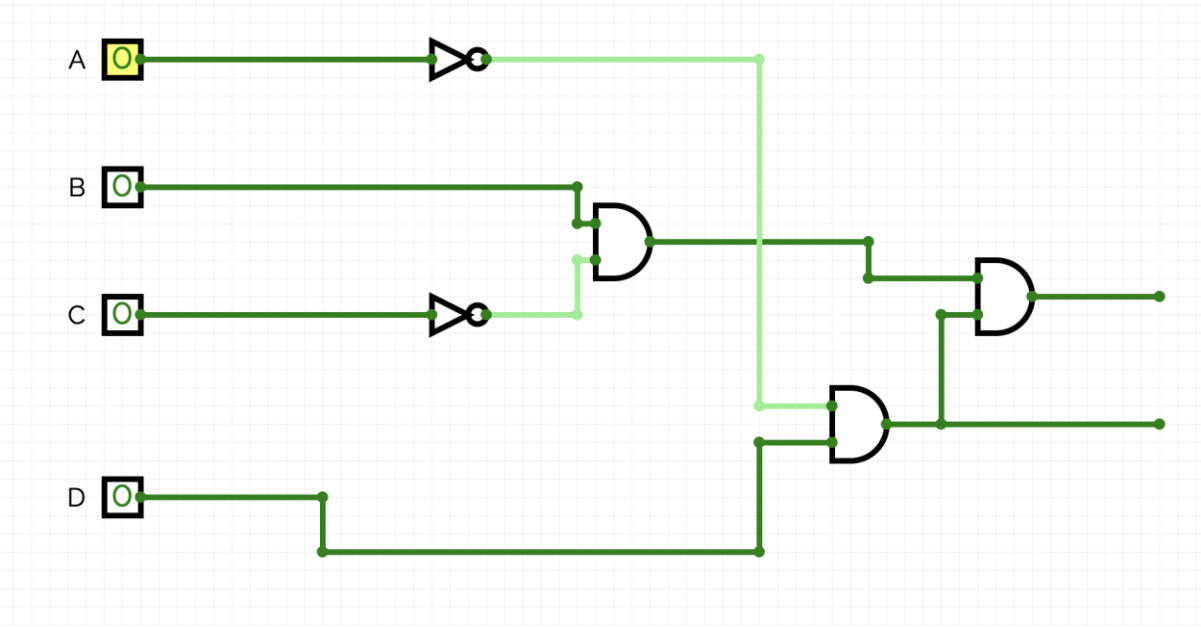
Y output = A’DC’+A’DB’

X output:

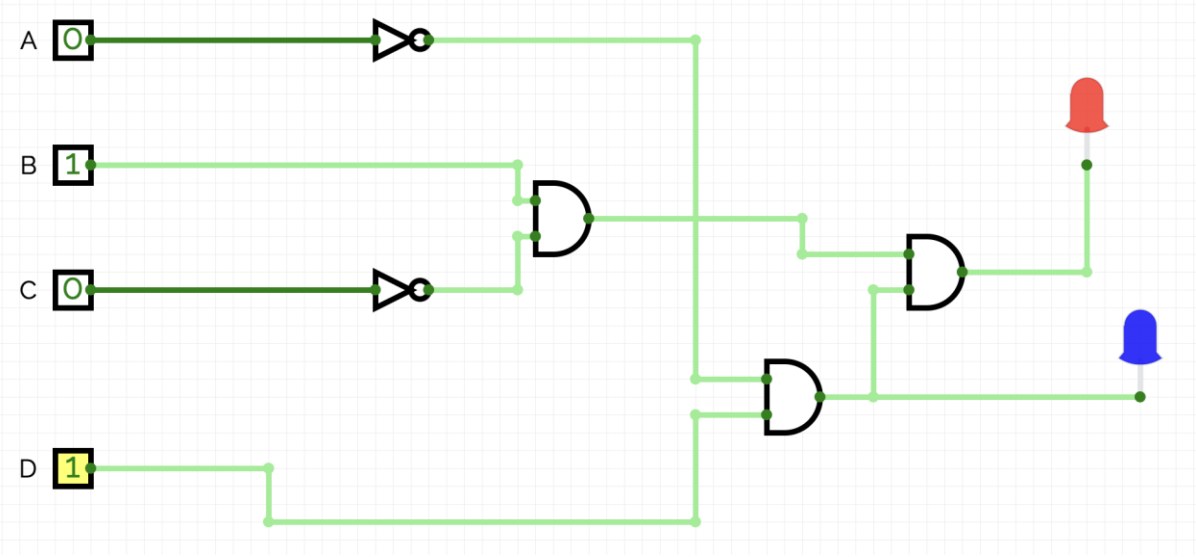
X output = A’BC’D

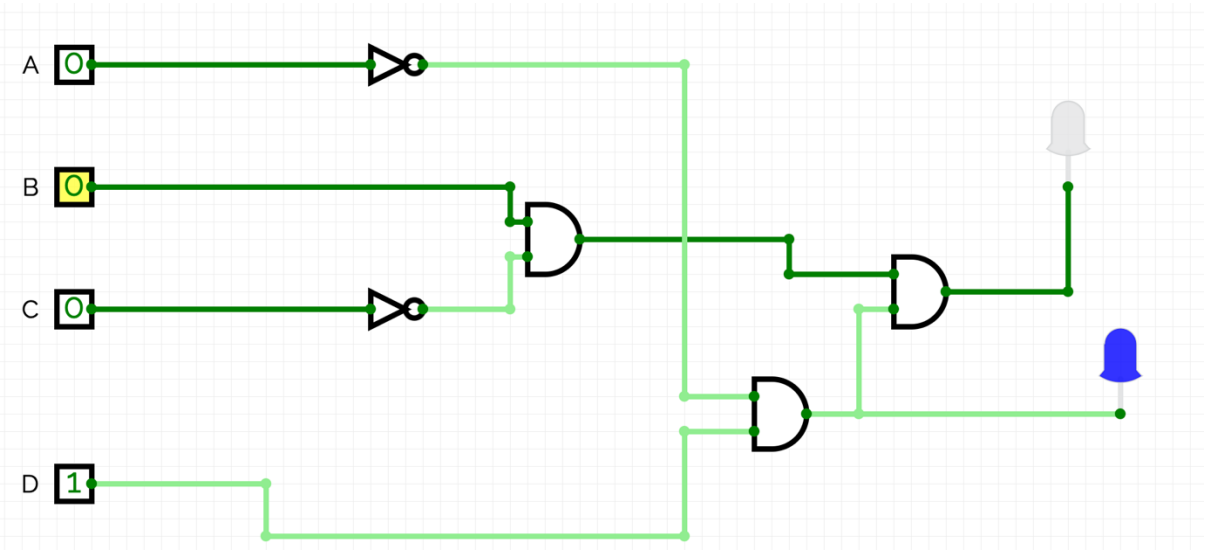
Designing a Safety Alert System using Logic Circuits

Logic Circuit



Simulated logic circuit

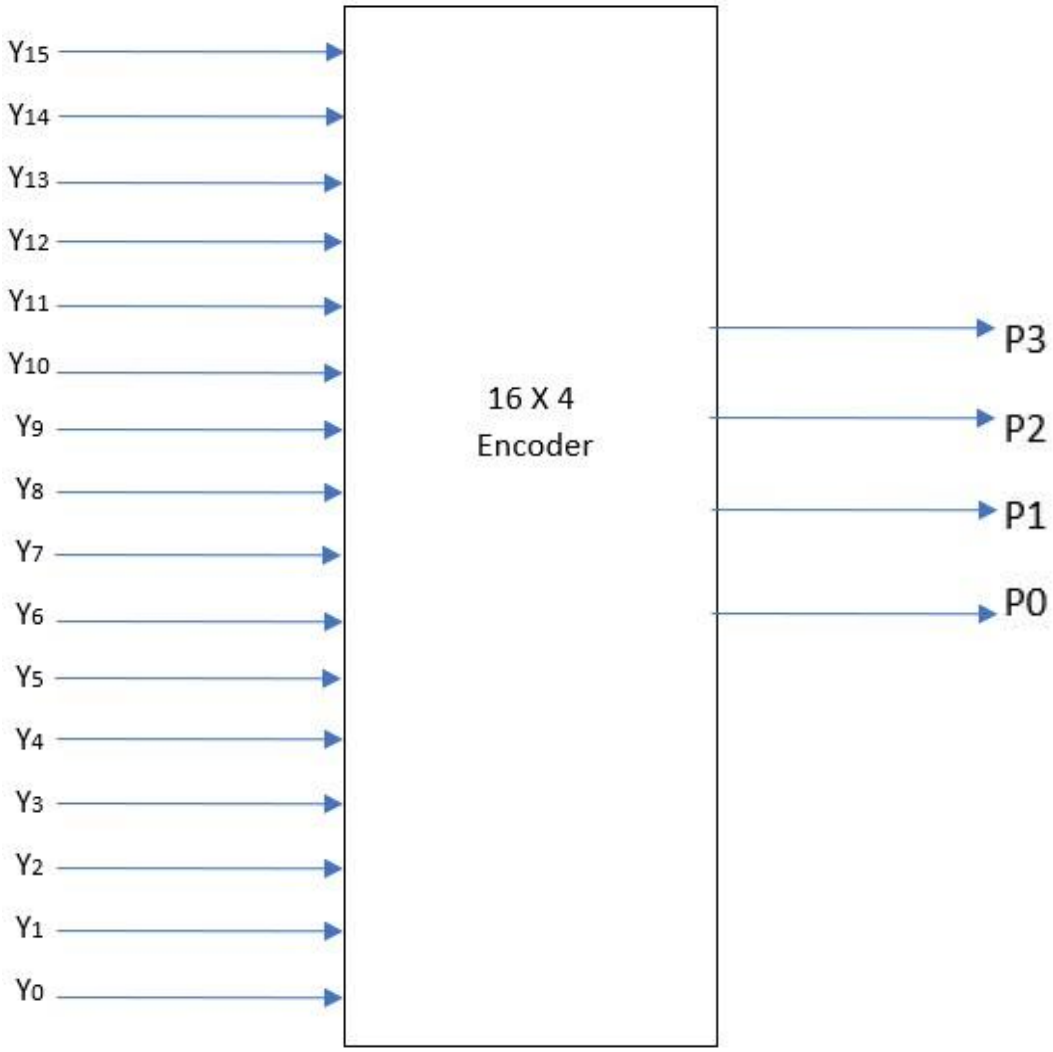



A
0
B
0
C
0
D
1

Question 2

Schematic diagrams and truth tables for encoder

Schematic diagram

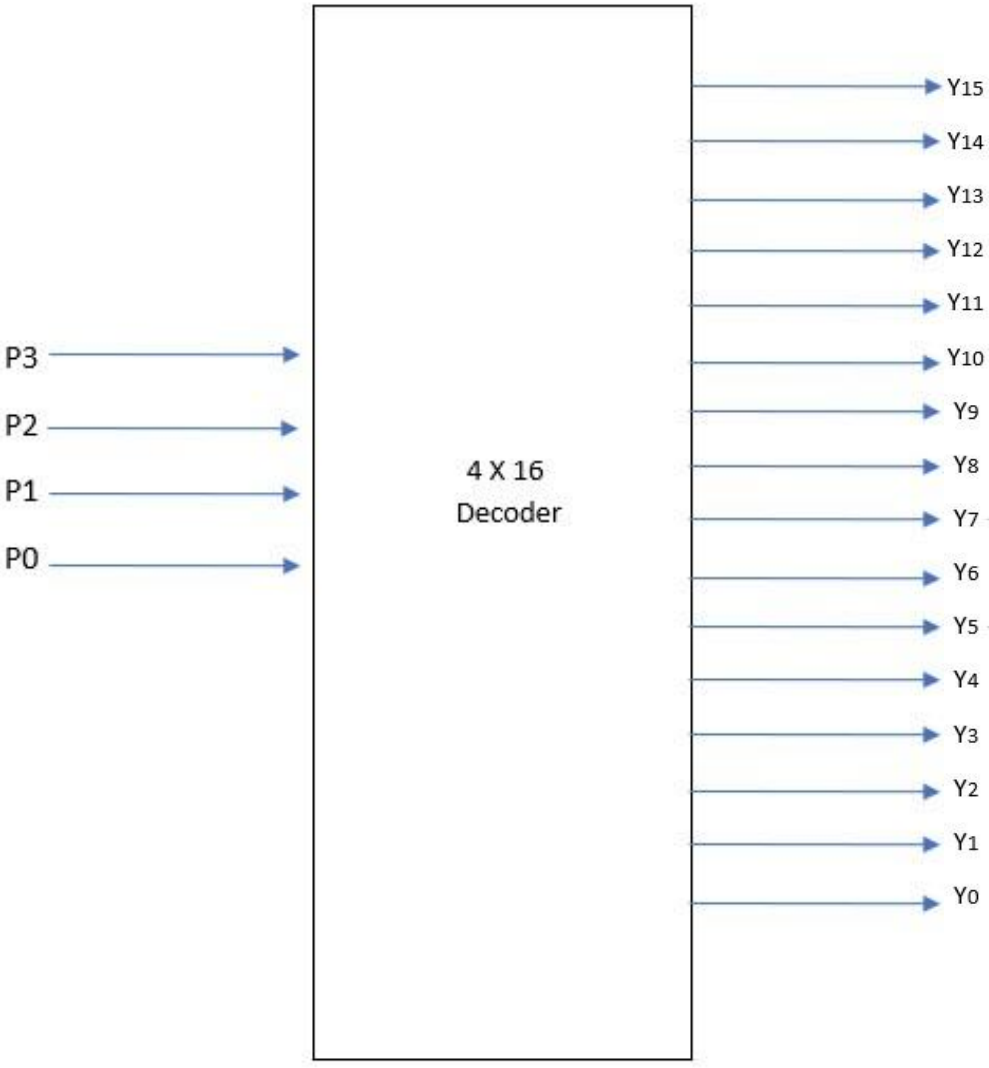


Truth table

INPUTS																OUTPUTS			
Y15	**Y14**	**Y13**	**Y12**	**Y11**	**Y10**	**Y9**	**Y8**	**Y7**	**Y6**	**Y5**	**Y4**	**Y3**	**Y2**	**Y1**	**Y0**	**P3**	**P2**	**P1**	**P0**
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	1
0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	1	0
0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	1	0	0
0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	1	0	1
0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	1	1	0
0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	1	1	1
0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	1	0	0	1
0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	1	0
0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1
0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0
0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	1
0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	0
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1

Schematic diagrams and truth tables for decoder

Schematic diagram



Truth table

INPUTS				OUTPUTS															
P3	P2	P1	P0	Y15	Y14	Y13	Y12	Y11	Y10	Y9	Y8	Y7	Y6	Y5	Y4	Y3	Y2	Y1	Y0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
0	1	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
0	1	1	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
1	0	0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0
1	0	1	1	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
1	1	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
1	1	0	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Simplify the expressions for the outputs in encoder and decoder.

Simplified expressions for the outputs in encoder.

$Y_3 = I_8 + I_9 + I_{10} + I_{11} + I_{12} + I_{13} + I_{14} + I_{15}$

$Y_2 = I_4 + I_5 + I_6 + I_7 + I_{12} + I_{13} + I_{14} + I_{15}$

$Y_1 = I_2 + I_3 + I_6 + I_7 + I_{10} + I_{11} + I_{14} + I_{15}$

$Y_0 = I_1 + I_3 + I_5 + I_7 + I_9 + I_{11} + I_{13} + I_{15}$

Simplified expressions for the outputs in decoder.

$Y_0 = I_0'. I_1'. I_2'. I_3'$

$Y_1 = I_0'. I_1'. I_2'. I_3$

$Y_2 = I_0'. I_1'. I_2. I_3'$

$Y_3 = I_0'. I_1'. I_2. I_3$

$Y_4 = I_0'. I_1. I_2'. I_3'$

$Y_5 = I_0'. I_1. I_2'. I_3$

$Y_6 = I_0'. I_1. I_2. I_3'$

$Y_7 = I_0'. I_1. I_2. I_3$

$Y_8 = I_0. I_1'. I_2'. I_3'$

$Y_9 = I_0. I_1'. I_2'. I_3$

$Y_{10} = I_0. I_1'. I_2. I_3'$

$Y_{11} = I_0. I_1'. I_2. I_3$

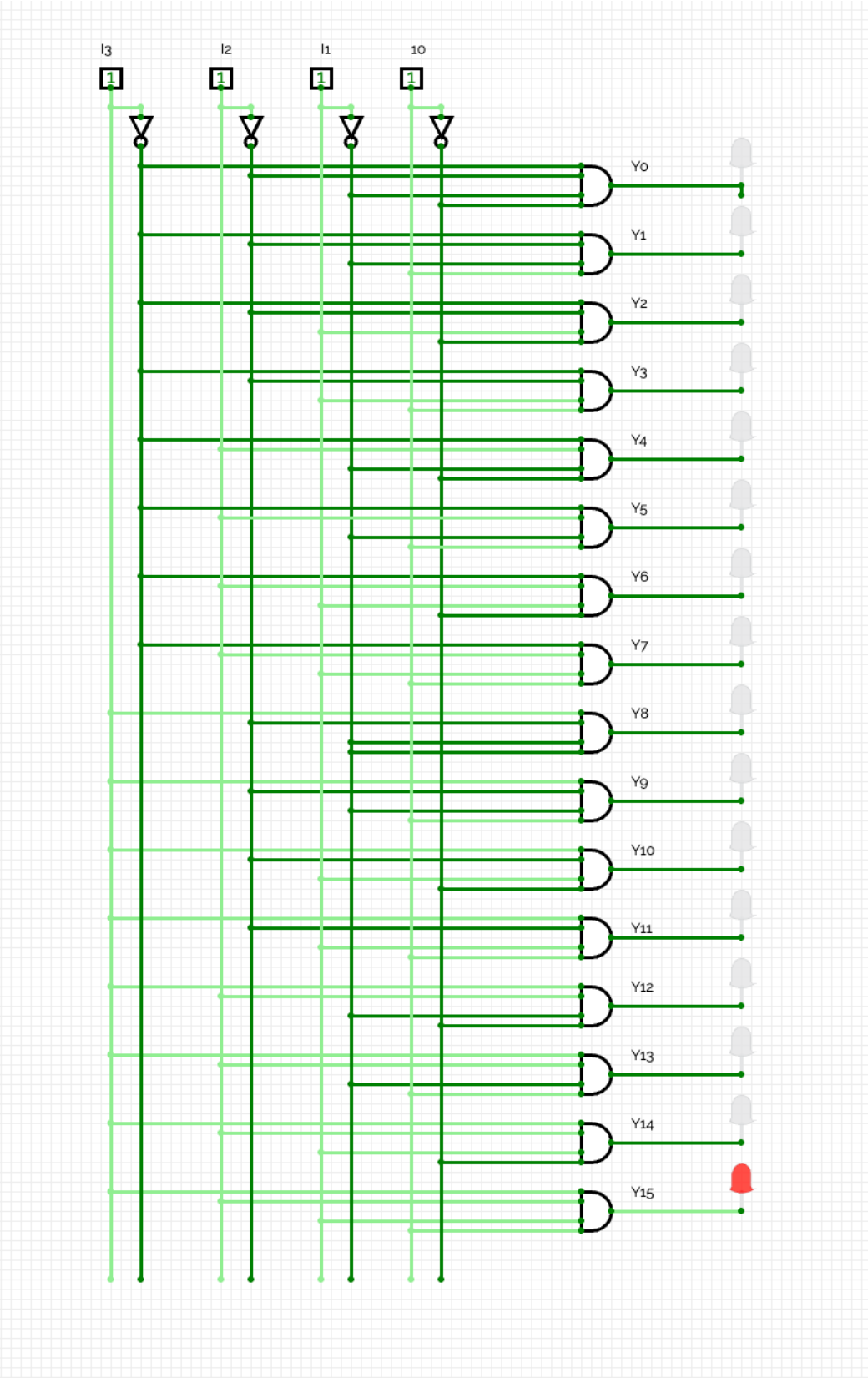
$Y_{12} = I_0. I_1. I_2'. I_3'$

$Y_{13} = I_0. I_1. I_2'. I_3$

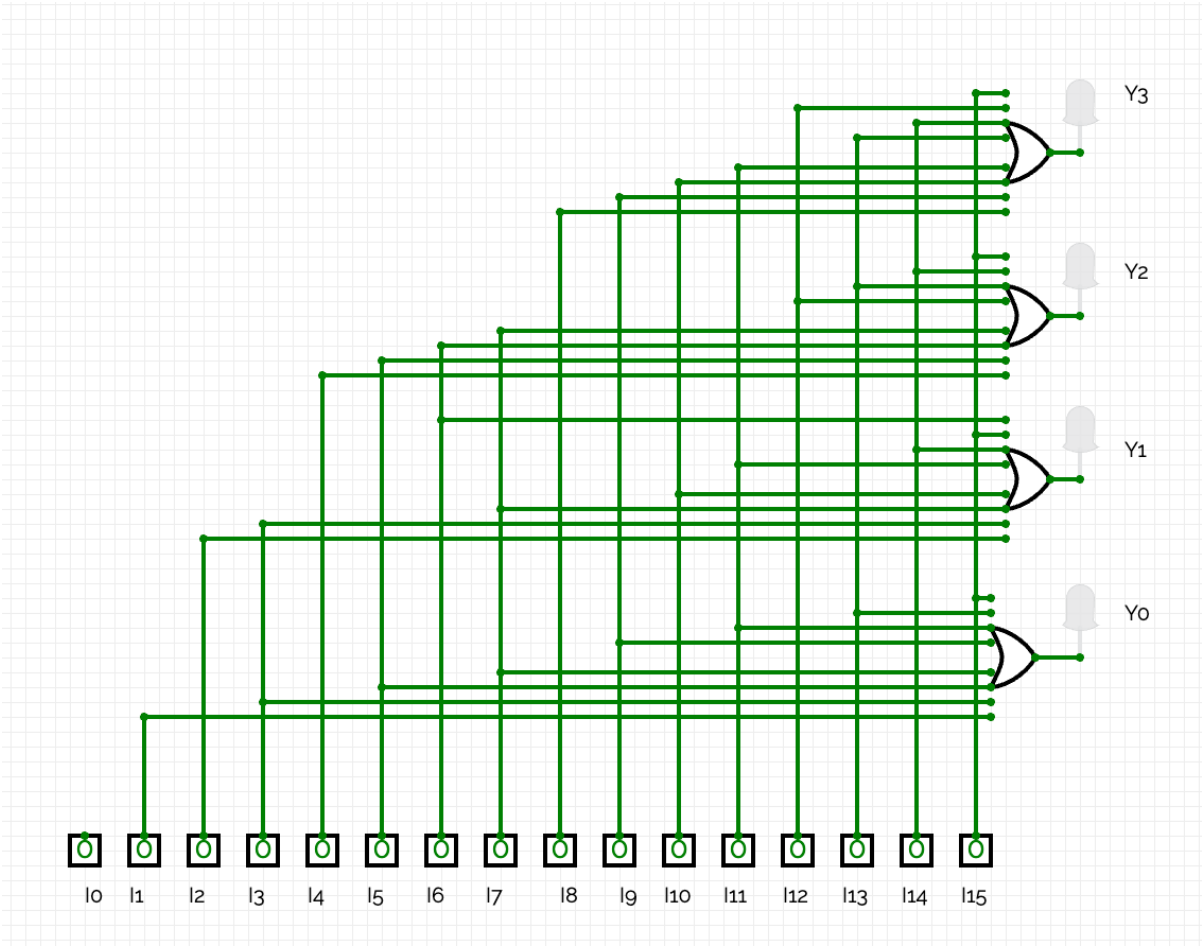
$Y_{14} = I_0. 1_1. I_2. I_3'$

$Y_{15} = I_0. I_1. I_2. I_3$

Simulate circuits for encoder and decoder.
Simulate circuit for decoder.

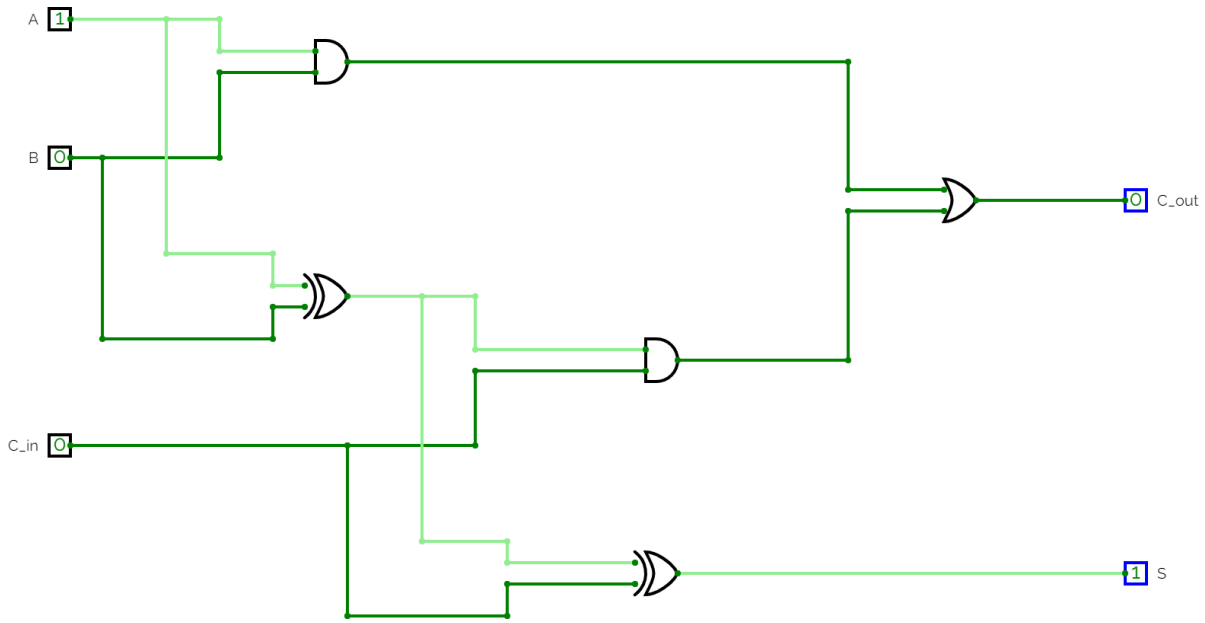


Simulate circuit for encoder.



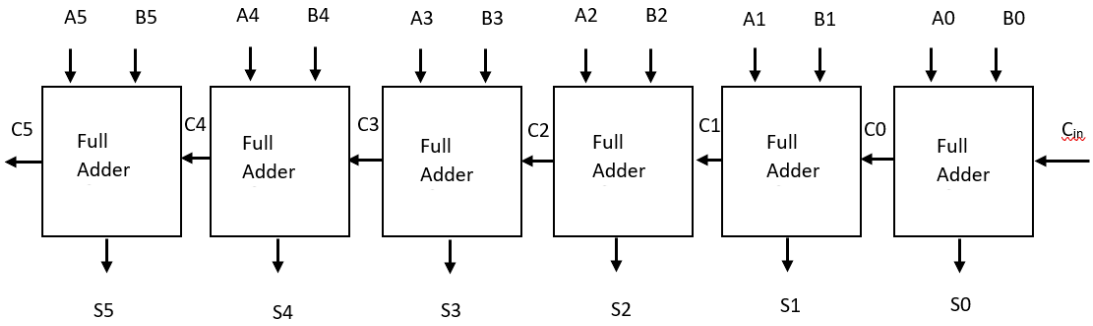
Question 3.

Full Adder can be implemented using multiple Half Adders. Using a suitable simulation, show how this could be achieved and explain how it works.



The circuits used in combinational logic gates are known as the full adder. Additionally, it adds a certain number of bits together. So that two 3-bit binary numbers with this can be added with this full adder. It denotes a pair of single-bit numbers that carry a carry bit. Using two half adders, we can create a full adder. There are 2 XOR gates, 2 AND gates, 1 OR gate, and 2 more gates in a full adder. Therefore, a full adder can be created by using 2 half adders, (by using OR, XOR and AND gates that are present in the half adder) with two inputs and three outputs respectively.

A set of Full Adders can be used to implement an adder which can add two 6-bit binary numbers. These types of adders are also called ‘Ripple Carry Adders’. Using a suitable simulation, show how this could be achieved and explain how it works.



Only the single-bit binary numbers can be added with the half adder since the full adder can only be used for adding 3-bit values. To add more than three bit binary numbers, we must utilize a ripple carry adder. The ripple carry adder allows us to add four bits and onwards. A six full adder is required to add two 6-bit binary values.

In order to obtain the sum as zero, the 1st two binary values should be added. We can then take the carry bit (0) to the following full adder and add the second two binary numbers to obtain the sum (1). Up until the completion of all 6 full adders, this procedure should be repeated. This is what a ripple carry adder does.